

MarginStream

A cross-margin derivatives venue with separated margin and matching authorities

SC6118 Scalable Systems Architecture for Fintech — Capstone, Group 1

LIU KEFAN ZHOU CONGXIANG WU YOUJHEN ZHANG HAOXI

1. Business context and requirements

1.1 The venue

MarginStream is a derivatives venue offering linear perpetual and dated futures on 40 underlyings, 120 contracts, to APAC retail and institutional clients, all margined against a single account balance rather than contract by contract. Leverage is capped at 20×; parameters below assume a median active account at 8×. The commercial reason for a unified account is capital efficiency — a client long one contract and short a correlated one should not fund both legs separately — so any design that quietly removes the offset removes the reason the venue exists, and the cost of every conservative step below is stated as a number.

Scale: 10^6 registered accounts, 10^5 with open positions, 10^4 whose positions or marks change between two issuances. A median account holds 5 contracts; the tail holds 50.

1.2 The requirement that does not decompose

The running case places pre-trade risk before the sequencer because the check is per-account and per-asset: freezing funds for one order says nothing about any other book, so it runs per connection and scales horizontally (Part 5 §1). Matching is sharded by symbol, single writer per shard (Part 2 §3).

A unified cross-margin account removes the first property and leaves the second. A fill on one contract changes the requirement of positions in other contracts held by the same account, sitting on shards being written concurrently. The invariant

the account's margin requirement must not exceed its equity

is global over the account and non-additive over contracts, yet it must be enforced before the order reaches the book, inside the same latency budget.

Locking the account, giving each holder a fixed sub-limit with no offset, and admitting optimistically are the three obvious resolutions; ADR-1 records why each fails.

§2 keeps matching exactly as the running case has it and moves the difficulty into a second authority: a margin allocator that runs off the order path and hands **each ingress gateway** a locally checkable share of the account's capacity. Matching shards hold no lease and make no margin decision.

Where this sits relative to the nearest candidate system. A real-time risk and liquidation engine for a margin venue is one of the five offered, and its hard parts are after the event: pricing the mark, ordering the waterfall, sizing the fund. Those are named out of scope here (§5.7) and this document does not claim them. The hard part taken instead is before the event, and it is a different one: a globally non-additive invariant enforced pre-trade over books that are sharded by symbol and written concurrently. A risk engine does not meet it because it assumes the account-level check has already been serialised somewhere; this design is about what happens when it cannot be.

1.3 Scope

In scope: the margin authority, the admission path, the degradation ladder, the liquidation and settlement path, and the audit trail for admission decisions.

Taken as given from the running case and cited rather than re-derived: the matching engine, the replicated log, determinism, and double-entry accounting. Out of scope: order routing, market making, fiat rails, wallet infrastructure.

1.4 Non-functional requirements

#	Requirement	Target	Consequence for the design
1	Order throughput	100k/s sustained, 1M/s burst	Admission is a local array operation; no allocator call on the order path

#	Requirement	Target	Consequence for the design
2	Admission-path latency	p50 < 20 μ s, p99 < 200 μ s, above matching	Per-account per-gateway scenario vector kept resident. Argued, not measured: E3 supports the scaling, not the target
3	Matching latency	p50 < 100 μ s, p99 < 1 ms in-engine	Unchanged; single writer per symbol
4	Margin correctness	Requirement never exceeds equity after any move the scenario grid covers	The closure of §2.4, with the factor of two shown tight
5	Tightening latency	Binds within the lease term under partition, immediately when the ordering point is reachable	Fence at the ordering point; §1.5
6	Allocator cadence	Issuance every 50–200 ms	§1.5, §1.6
7	Allocator throughput	$\approx 3 \times 10^7$ scenario operations per issuance	Sharded by account; grid evaluated as a vector
8	Availability	99.99% for order entry; failover < 3 s	Replicated log, as in the running case
9	Degradation	The venue can bound its own loss in every state above HALT. Client-initiated risk reduction is not preserved under partition	Venue-initiated liquidation (§5.4); §3.3 states what is given up
10	Auditability	Target: every admission <i>and refusal</i> replayable with the figures it compared. Today: admissions only. A refusal never reaches the ordering point, and §2.6's gateway refusal journal is not built	
11	Solvency	Target: Σ user liabilities $\leq \Sigma$ venue assets, continuously checkable. Today: three facts about one account (§4.3). The ledger is unimplemented and the fund unsized, so the venue-level claim is an argument	

Rows 1, 3 and 8 are inherited; 2, 6 and 7 derived in §1.5–1.7; 4, 5 and 9 established in §2, §3 and §5. Rows 10 and 11 are targets, marked as such; §9.4 lists everything designed and not built.

1.5 What sets the lease term

A lease is a fixed ceiling a gateway may spend for the length of its term. The term is the mechanism's principal operational trade-off parameter — the scenario grid, the add-on parameters, the price bands, the fee caps and the gateway weights are all chosen too — and it is the one bounded from both sides at once.

Recompute cost is the floor. §1.6 works out $\approx 3 \times 10^7$ operations per issuance for the whole book, which is an ordinary workload at a 100 ms cadence and is not one at 1 ms.

Tightening latency is the ceiling. A gateway that cannot be reached keeps admitting inside its ceiling until the term ends, and no allocator message changes that — which is what the term is for.

The lease term is the worst-case enforcement latency of a **tightening** decision **under partition**.

Raising a limit takes effect at the next issuance, and so does lowering one when the allocator can reach the gateways; the term bounds only the unreachable case, where it bounds everything — collateral cut, downgrade, withdrawal. 50–200 ms is a **design target chosen against an assumed tightening SLO**, not a figure derived from any supervisory deadline.

One path does not wait: fencing the lease at the ordering point (§2.5), which is immediate and needs no gateway to be reachable. The term bounds *routine* tightening, where fencing every affected lease is a heavy instrument.

An earlier version issued capacity as a schedule contracting with a published market state. It is withdrawn: see ADR-2 and Appendix A.

1.6 What the allocator has to compute, and how often

Per account per issuance: one evaluation of the scenario term at $|S| \times \text{contracts} \approx 16 \times 5 \approx 80$ multiply-adds, one feasibility check of §2.4's condition at the same order, and a bisection for the scale over 10^9 minor units at ≈ 30 iterations — $\approx 3 \times 10^3$ operations in all. At 10^4 accounts changed per issuance that is $\approx 3 \times 10^7$ **per issuance** and, at a 100 ms cadence, $\approx 3 \times 10^8$ **per second**. Sixteen allocator shards (§2.6) carry $\approx 2 \times 10^7$ each: headroom, not a fit.

No invariant spans two accounts, which is what makes the sharding trivial, and recompute is incremental: unchanged positions and marks keep their ceilings.

1.7 What the admission path may do per order

The admission decision must not recompute the requirement from positions. Each gateway keeps, per account, the running loss numerator under each of the $|S|$ scenarios plus the worst-fill gross and debit totals, updated on each order state change. Admitting is one pass over the grid, one symbol's gross update and three integer comparisons — independent of how many orders are live.

Memory is ≈ 128 B per (account, gateway) pair; at 10^5 active accounts touching 5 gateways each, $\approx 5 \times 10^5$ pairs and ≈ 64 MB resident.

Measured, not argued. E3 times the incremental path against a full scan computing identical envelopes, after checking the two agree on 400 random books. From the recorded run in `results/e3_hot_path.json`: increasing live orders 10×, from 50 to 500, changed the incremental median by 1.1%, while the full scan over the same range grew 6.7×; widening the grid from 7 scenarios to 16 raised the incremental median by 34%. That is $O(|S|)$ against $O(\text{orders} \times |S|)$.

The absolute figures are deliberately not quoted here. They are wall-clock nanoseconds from CPython on a shared machine, they differ by a third between the hosts this has run on, and they are three orders of magnitude from what a compiled implementation would need. Only the ratios above are evidence, and they support the scaling claim rather than row 2's latency target, which remains argued. `results/PROVENANCE.md` records the machine, and `REPRODUCE.md` lists other hosts under the host that produced them.

1.8 What these numbers commit us to

- Three ceilings per gateway per account, checked against absolute figures rather than increments, because an incremental charge does not bound the account's requirement (§2.4).
- An allocator partitioned by account, ≈ 16 shards at this scale.
- A lease term chosen from recompute cost below and tightening latency above, with fencing at the ordering point as the path that does not wait for it.

Each is revisited in §7 with the alternative that lost.

2. Architecture

2.1 Topology

Figure 1 gives the component view. Orders reach the venue through N ingress admission gateways, pass through a single **ordering point**, and are forwarded to a matching core partitioned by symbol. Matching keeps a single-writer total order per symbol and no order crosses a shard; that is the running case's arrangement and this design does not touch it.

What is added is a second authority. The **margin allocator** runs off the order path, computes each account's capacity, and hands each gateway a locally checkable share of it. A gateway decides in constant time from what it already holds; it calls no allocator and reads no other gateway's state.

	Partition key	Why
Matching	Symbol	Price-time priority is a total order per book, so a book must have one writer
Allocator	Account	Margin is an account-level quantity, and no invariant spans two accounts

The partitionings are orthogonal, which is the structural point: an account's positions spread across matching shards, a shard holds many accounts, and neither needs the other's partition to be correct. The single-writer core is plural — one writer per symbol for the book, one per account for the capacity.

2.2 Why a lease exists at all

Admission happens at N gateways upstream of the shard, so either every order pays a round trip to a shared counter or capacity is divided into locally checked shares. **The value of a lease is upstream early shedding** — stopping a burst before it queues at the single-writer shard — not replacing a counter inside it. Collapsed to one gateway the mechanism reduces to a local counter, and we would say so rather than defend the complexity.

2.3 What can be divided, and what cannot

Write the account's requirement as a scenario term plus an add-on. Two gross figures are needed and they are not the same object:

```

R(P)      = max over k in S of  loss_k(P)
G_k(P)    = sum_s |q_s| * mark_s(k)          gross at the marks of scenario k
G+(P)     = sum_s |q_s| * max_j mark_s(j)     gross at the highest mark in S
M_k(P)    = R(P) + A(G_k(P))                what the account owes at k
A          = phi, convex, non-decreasing, phi(0) = 0

```

G_k is what the requirement is computed from; G_+ is what a lease reserves against, because the marks move during a term (§2.4). $G_k(P) \leq G_+(P)$ for every k by construction. The code keeps them apart as `gross` and `gross_reach`.

R is the worst loss across a fixed scenario set S , and the loss under any single scenario is linear in positions. A is a concentration and liquidity add-on.

Model boundary. The algebra below holds for any finite S . The set used in every correctness experiment here is **seven points on a single factor with non-negative loadings**; E3 additionally times a 16-point grid. Nothing here shows that a single-factor grid is adequate for 40 underlyings — basis and idiosyncratic risk would need more factors or a wider set, and the evidence for *that* choice is not in this document. What is shown is that the decomposition, the closure and the lifecycle are correct for whatever finite S is picked.

The partition is by gateway, not by symbol. Two gateways can hold opposite positions in the *same* symbol, and those net inside the account, so gross is not additive across the partition.

Lemma 1 — R is sub-additive. $R(P) = \max_k \sum_g \text{loss}_k(P_g) \leq \sum_g \max_k \text{loss}_k(P_g) = \sum_g R(P_g)$: a single scenario cannot beat the per-gateway worst cases taken separately.

Lemma 2 — gross is sub-additive. Per symbol, $|\sum_g q_{\{g,s\}}| \leq \sum_g |q_{\{g,s\}}|$ by the triangle inequality; multiplying by a positive mark and summing gives $G_k(\sum_g P_g) \leq \sum_g G_k(P_g)$ for every k , and the same for G_+ , with equality only when every gateway holds the same sign in every symbol.

Lemma 3 — A does not decompose. A convex function through the origin is super-additive on non-negative arguments, so $\sum_g A(G_g) \leq A(\sum_g G_g)$. Per-gateway add-on allowances added up under-state the whole, however the positions are split.

The decomposition rule follows rather than being chosen:

The sub-additive parts divide into per-gateway ceilings checked locally. The add-on does not; it is evaluated once, centrally, on the summed gross, and A being non-decreasing is what makes that an upper bound.

Chained, for the realised scenario k :

$$G_k(P') \leq G_+(P') \leq \sum_g G_+(P'_g) \leq \sum_g \lambda_g^G G$$

the first step by construction, the second by Lemma 2, the third by the admission rule. A non-decreasing then carries it to $A(G_k(P')) \leq A(\sum_g \lambda_g^G G)$. Nothing in that chain needs gross to be additive. Lemmas 1 and 3 are checked over 2,000 sampled portfolios in `tests/test_algebra.py`, in the predicted directions.

2.4 What a lease grants

A lease cannot undo an admission it has already granted. Everything here follows from that.

Three envelopes

Envelope	What it bounds	Why separate
λ_g^R	worst-fill scenario requirement of everything the gateway holds	sub-additive, so it divides
λ_g^G	worst-fill G_+ the gateway can reach	an order can lower R while raising gross its effect on equity is covered by neither the scenario requirement nor the gross add-on
λ_g^D	execution cost the gateway can still incur, plus cost already incurred that the equity the current lease was solved against does not yet reflect	

The three are not three allocations: λ^G and λ^D are issued at fixed ratios to λ^R , so the solver searches one scalar along a ray through a three-dimensional feasible set. Dropping the second half of λ^D is what made capacity decay every term in an earlier implementation (d7).

What a gateway holds is orders, not positions Two resting orders of opposite sign net to nothing, and if only one fills the account carries the other side. The envelopes are taken over the worst subset of fills that could still occur, which needs no enumeration because the loss under a fixed scenario is linear:

```
E_k = loss_k(filled) + sum_i max(0, loss_k(order_i))
R_wf = max(0, ceil(max_k E_k / DEN))
G_wf = sum_s max_j mark_s(j) * max(|filled_s + buy_s|, |filled_s - sell_s|)
```

Both closed forms agree with enumeration of all 2^n fill subsets on 4,000 random books (`test_worst_fill_exhaustive`). Admission compares these **absolute** figures against the ceilings, not the increment an order adds: a leg flipped from short to long leaves the increment unchanged while the account's requirement moves to its maximum (c1).

Where gross is measured A lease reserves against G_+ , not against the gross standing when the solve ran. A short position's adverse scenario raises the mark, raises gross and raises the add-on, while a figure measured at the issuance mark does not move. Reserving at the issuance mark admits 296 lots in the worked case of m1 and finishes 382,143 above equity; reserving at G_+ admits 249 and finishes 5,842 inside. ADR-3 gives the cost and the scope of the tightness claim.

The condition and its closure

$$2 * \sum_g \lambda_g^R + A(\sum_g \lambda_g^G) + \sum_g \lambda_g^D \leq E_0$$

with $E_0 = \text{Collateral} + \sum_s (\text{cash}_s + q_s * \text{mark}_s) - \text{fees}$, mark-to-market equity rather than collateral. There is no market state in it.

Write P' for the position the term ends with, D for the execution cost it incurred, k for the realised scenario. Three bounds come from the admission rule and the fourth because R is a maximum over a set containing k :

$$\begin{aligned} R(P') &\leq \sum_g \lambda_g^R & G_k(P') &\leq \sum_g \lambda_g^G \\ D &\leq \sum_g \lambda_g^D & \text{loss}_k(P') &\leq R(P') \end{aligned}$$

Adding requirement, cost and realised loss:

$$M_k(P') + D + \text{loss}_k(P') \leq 2 \sum_g \lambda_g^R + A(\sum_g \lambda_g^G) + \sum_g \lambda_g^D \leq E_0$$

and since $\text{Equity_after}(k) = E_0 - D - \text{loss}_k(P')$,

$$M_k(P') \leq E_0 - D - \text{loss}_k(P') = \text{Equity_after}(k)$$

Subtracting D is why the third resource exists. A conclusion of $M \leq E_0 - \text{loss}$ would leave execution cost out of the arithmetic while still listing λ^D as a ceiling.

The factor of two is a closure, not a margin, and it is tight. Set $A = D = 0$ and take $R(P') = \lambda^R$ with the realised scenario attaining the maximum, so $\text{loss}_k = R$. Then $M_k = \lambda^R$ and $\text{Equity_after} = E_0 - \lambda^R$, equal precisely when $E_0 = 2\lambda^R$. With $c < 2$ the solve issues $\lambda^R = E_0/c$, and the same position ends with $M_k = E_0/c > E_0 - E_0/c$. The coefficient cannot be reduced.

What the closure costs Utilisation is capped near half of equity before the add-on reserve. In E1's binding trial — every order filled at the worst price and fee the policy allows — the risk and debit envelopes reach 99% and the requirement is 49% of equity, with no breach. The offset decomposition gives up is separately §2.3's sub-additivity gap (§7).

2.5 Ending authority

Every lease carries `(account, epoch, generation, lease_id)` and is registered at the ordering point against the account, the holder and the authority kind (Appendix C.3). A holder is `(gateway_id, incarnation)`: a process that restarts and reuses its identity is a different holder, and both may be live at once.

Two quantities are tracked per holder and only one expires. **A term ends authority to admit; it never ends the exposure already created.** A clock comparison does not prove a partitioned holder has stopped — its clock may be behind. **A fence at the ordering point** does, because nothing reaches a book except through it, and it need not reach the gateway: E7 fences without telling any gateway, the ordering point turns away 50 submissions, and the run is otherwise identical.

A fence is not terminal for exposure either: a resting order can still fill. Removing its reservation needs a cancel acknowledged at the ordering point; lowering a holder's recorded exposure needs a terminal reconciliation with that lease's seal, or an account-wide barrier. §5.4 is the full lifecycle.

When the solve is infeasible, leases are issued in **quarantine** and the gateway admits nothing — including orders that look locally like risk reduction, because an order lowering one gateway's requirement can raise the account's by removing a hedge held elsewhere (c9).

2.6 Components

Component	Holds	Decides
Ingress gateway	three ceilings per account; worst-fill running totals over the grid	admit or refuse, by comparing absolute figures against ceilings. Its state is a deterministic fold of the log, so a snapshot bounds replay time rather than being a correctness requirement
Ordering point	the log; the lease registry; sessions; fences and seals	that an admission is the next number for a live, correctly bound lease; that a fill matches its recorded terms; that a basket commits as one record. Writes nothing when it refuses
Margin allocator	committed exposure per holder; generations; credit versions	the condition of §2.4, once per account per issuance. Sharded by account, ≈ 16 shards
Liquidator	the merged account view	which basket to transfer, checked against the merged account rather than a ceiling — the check c9 shows a gateway cannot make. Inside the trusted computing base on its own account (§6.1)
Market-data publisher	marks	nothing on the admission path, but marks set equity, the scenario displacements and $G+$, so this path fixes how much capacity is solved for. Named here, not designed
Matching core	books	unchanged; applies a client order ID at most once
Ledger	postings	double-entry, append-only. A lease never appears in it (§4.3)

2.7 What is on the replicated log

Every admission with its lease, sequence number, holder and terms; every fill with price and fee; every cancel acknowledgement; every fence with its seal; every liquidation basket as one record; every settlement barrier; and the lease inputs per account per issuance.

Three components reconstruct the same facts from it without reaching each other: a recovering gateway rebuilds its order state, the ledger rebuilds the account, and the allocator computes occupancy for a holder that may be gone.

Not on the log: the per-gateway ceilings, which cost ≈ 32 MB/s against an ≈ 12.8 MB/s order stream to carry a value each gateway derives from ≈ 8 MB/s of inputs (ADR-4 has the arithmetic). Also not on the log: the scenario vectors and running gross, caches of pure functions of the order state.

Batching, and what it costs here. The running case replicates a batch of sequenced commands rather than one at a time, because replication cost is per-round-trip and amortises (Part 4 §3). The same applies to admissions, and it interacts with two things this design added. A batch is committed or not as a unit, so the gap-free per-lease sequence survives batching only if the ordering point assigns numbers at batch entry and refuses the whole batch on any gap — partial commit would leave a hole that a seal later claims to cover. And a fence arriving mid-batch must take effect at the batch boundary, not inside it: admitting the first half of a batch under a lease fenced before the second half would produce admissions after the seal's terminal sequence. Both are resolvable — number at entry, fence between batches — and neither is implemented; the simulator commits one admission at a time, which is the worst case for replication cost and the simplest case for the argument.

2.8 What this architecture does not do

It does not identify who is behind an order, price liquidity, or make the matching core elastic. And it does not remove the need for a liquidation waterfall: it provides the trigger, the fencing and the unwind, and leaves who absorbs a shortfall to a design this document does not contain.

2.9 Architecture diagrams

The three views §2 requires — component, data flow, deployment — plus the two the failure and degradation stories need. Mermaid source; `paper/DIAGRAM_EXPORT.md` covers rendering.

Figure 1 — Component view

Two authorities partitioned on different keys, and one ordering point that both of them and every recovery path read from. The liquidator is drawn apart from the gateways because its orders are checked against the merged account rather than a ceiling, and its transfers do not touch the order books.

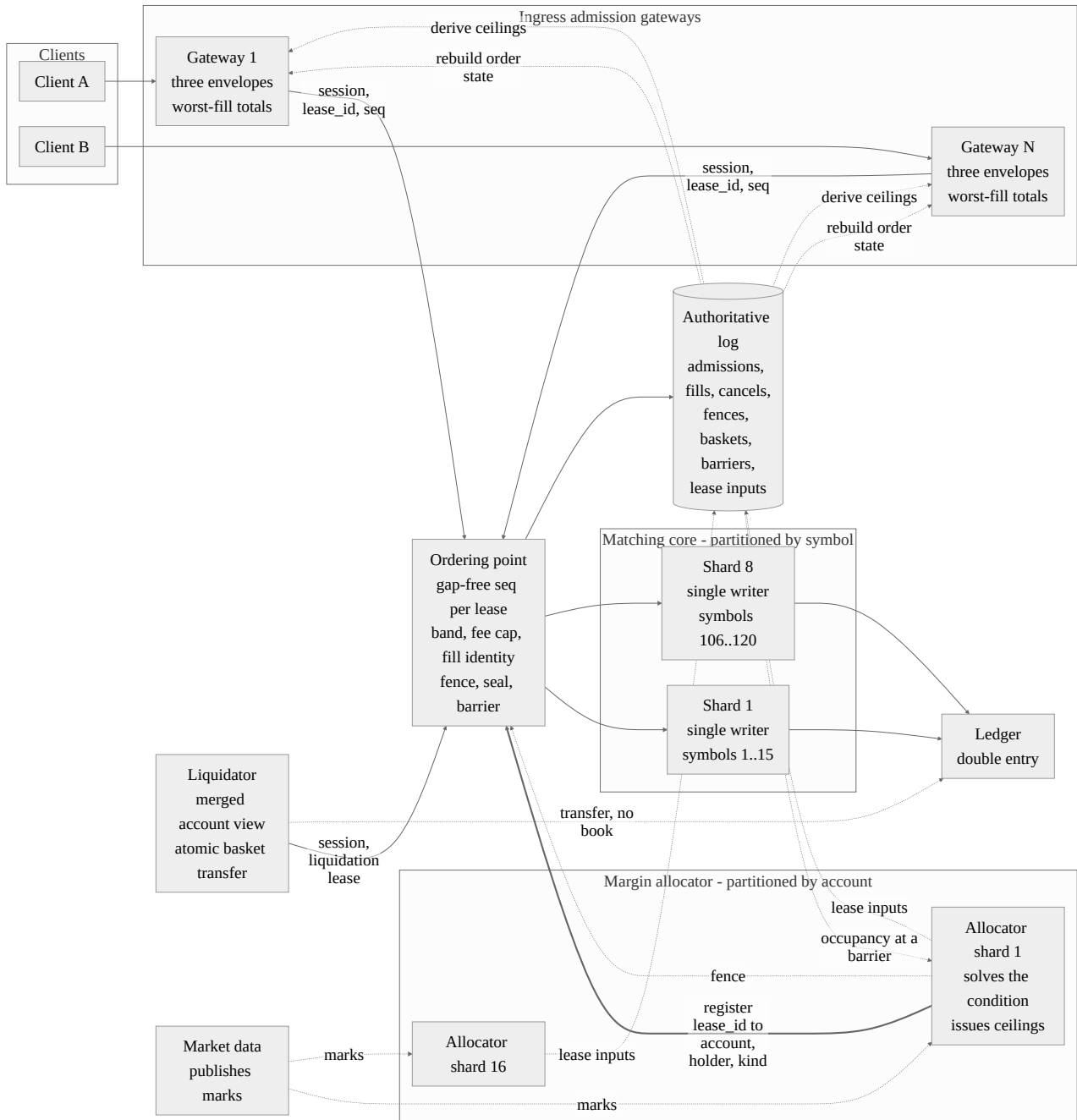


Figure 1: Component view. Two authorities partitioned on different keys, and one ordering point both of them and every recovery path read from. The thick edge is the lease registration this design would be unsound without.

Solid edges are the order path; dashed edges are asynchronous. The allocator never reads a gateway: every figure it acts on comes from the log. The thick edge is the one this design would be unsound without — a `lease_id` alone is a bearer token, and the ordering point knows which account, holder and authority kind it belongs to only because the single issuer registers the binding.

Figure 2 — Data flow for one order

Three stages. The first two are the gateway's and nothing downstream re-derives them; the third is the ordering point's and a compromised gateway cannot influence it. Refusals are collapsed to one node per stage.

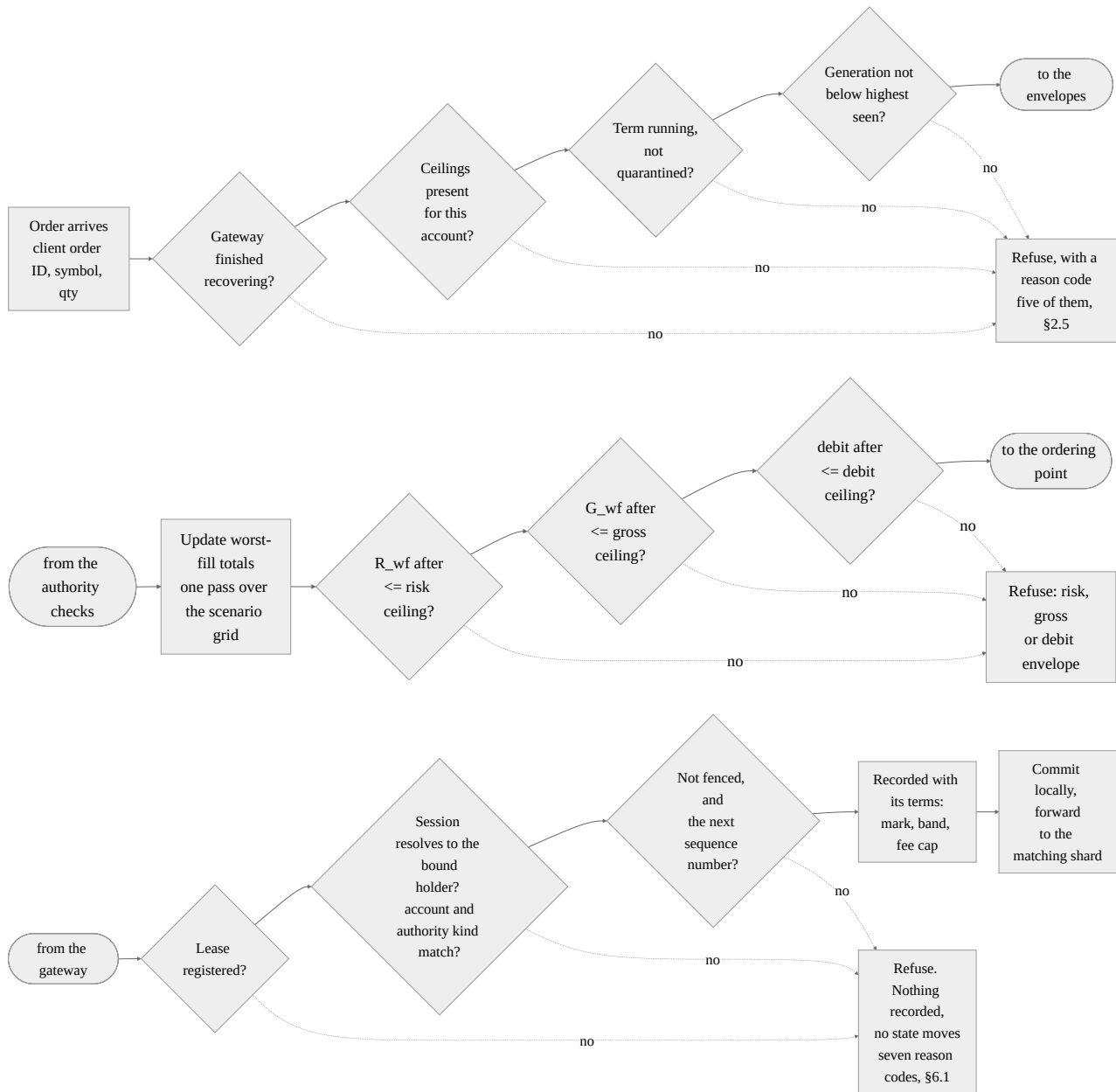


Figure 2: Data flow for one order, in three stages. The first two are the gateway's and nothing downstream re-derives them; the third is the ordering point's and a compromised gateway cannot influence it. Refusals are collapsed to one node per stage; the reason codes are listed in the sections cited.

The whole per-order margin cost is the one pass over the scenario grid in stage two: the running totals are updated per order state change rather than recomputed, so admission is flat in the order count and linear in the grid width (E3).

Figure 3 — Liquidation and settlement

The path from a shortfall to capacity being returned, in three stages. E7 injects a fault at each.

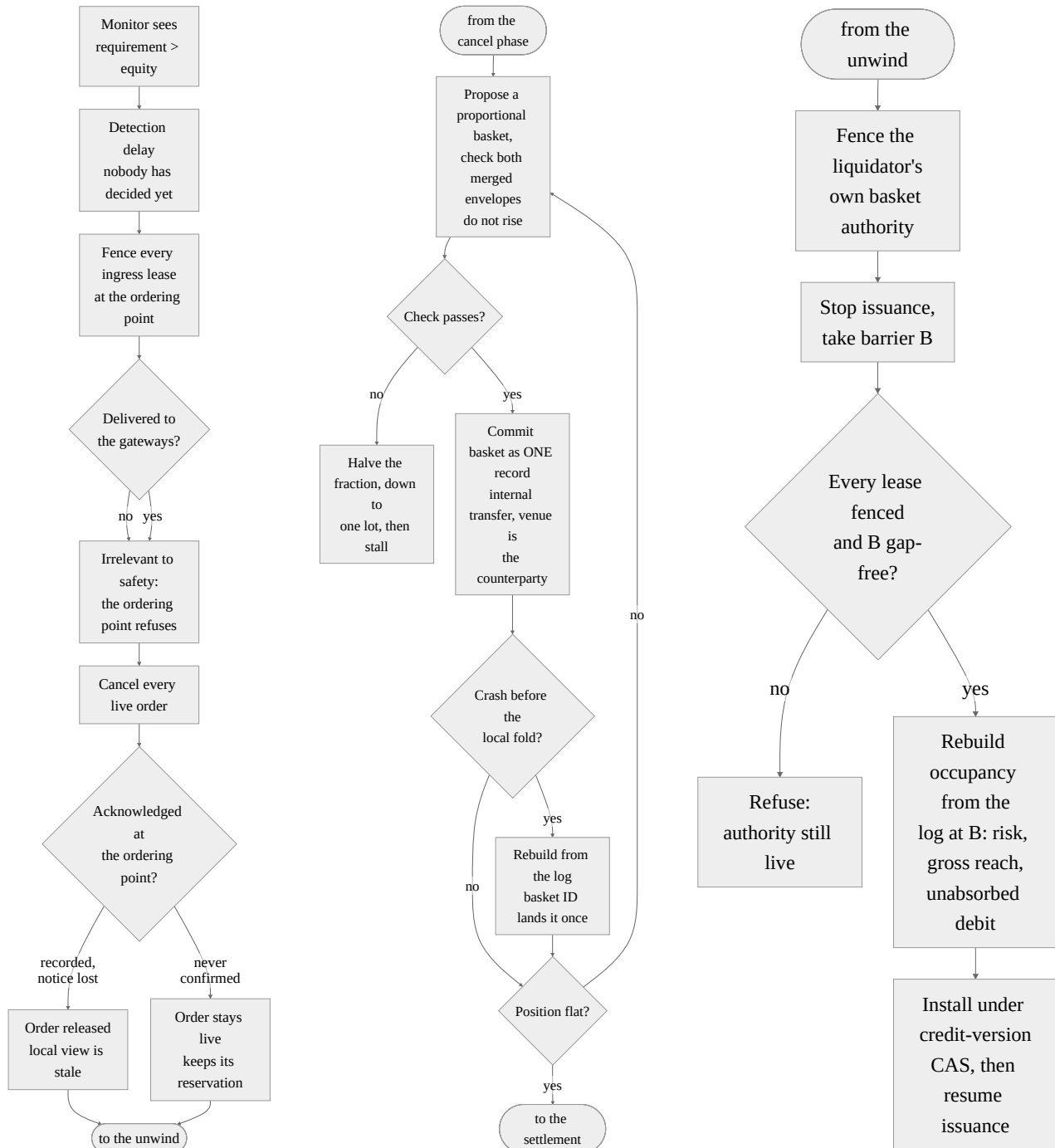


Figure 3: Liquidation and settlement. Stop, unwind, settle. The two exits from the cancel check are the two different facts of §5.4.

The cancel check has two labelled exits because they are two different facts (§5.4): a cancel recorded at the ordering point releases the order, one the matching side never confirmed does not. The barrier refuses on `no_fence` and on a live liquidator, and E7 exercises both refusals.

Figure 4 — Failure paths and the degradation ladder

Every transition is labelled with what causes it and what the venue still accepts in that state. A state a previous version called REDUCE_ONLY has been removed: a gateway does not accept locally-judged risk-reducing orders, because c9 shows that judgement is unsound across gateways.

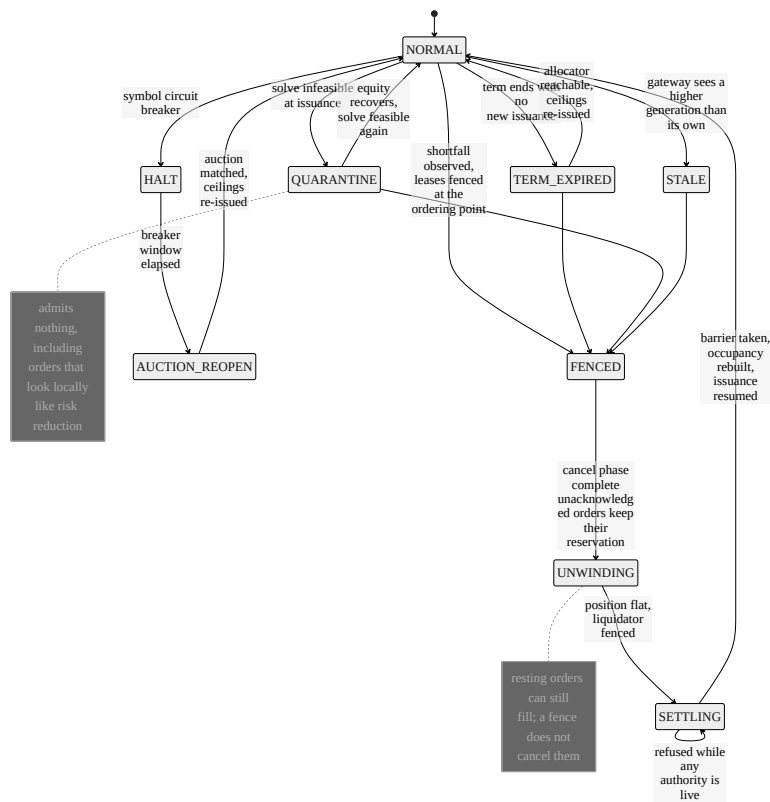


Figure 4: Failure paths and the degradation ladder. The four edges into FENCED are the four ways this design loses authority, and all four fail closed for new risk.

The four edges into FENCED are the four ways this design loses authority, and all four fail closed for new risk. Risk reduction happens on the liquidation path in every one of them, which is the CAP position §3.3 defends and the correction that removed REDUCE_ONLY.

Figure 5 — Deployment view: target, not built

Everything measured in this document runs in a single process against an in-memory ordering point. **No part of this figure has been implemented or measured.** It is drawn because the architecture is not complete without saying where each component lives and what replicates it, and it is labelled because a diagram of unbuilt infrastructure next to measured results otherwise invites the reader to give both the same standing.

Three things here are load-bearing and unbuilt, and they are the three §5.7 lists. The ordering point is replicated, and every claim about fencing, sealing and barriers assumes it survives a node loss without losing or reordering the log. The allocator has no snapshot or failover: it is the only component that cannot rebuild itself from the log. And the liquidator is venue infrastructure, which is what makes its transfers internal (§5.4) and what puts it inside the trusted computing base (§6.1).

What the recovery evidence supports is the property replication needs — each component’s state is a deterministic, idempotent fold of one ordered log — and not that the replication works.

3. Consistency map

3.1 Per flow

One row per flow, with the model chosen and why it is survivable. The test is “weakest the business can survive”, not “strongest available”.

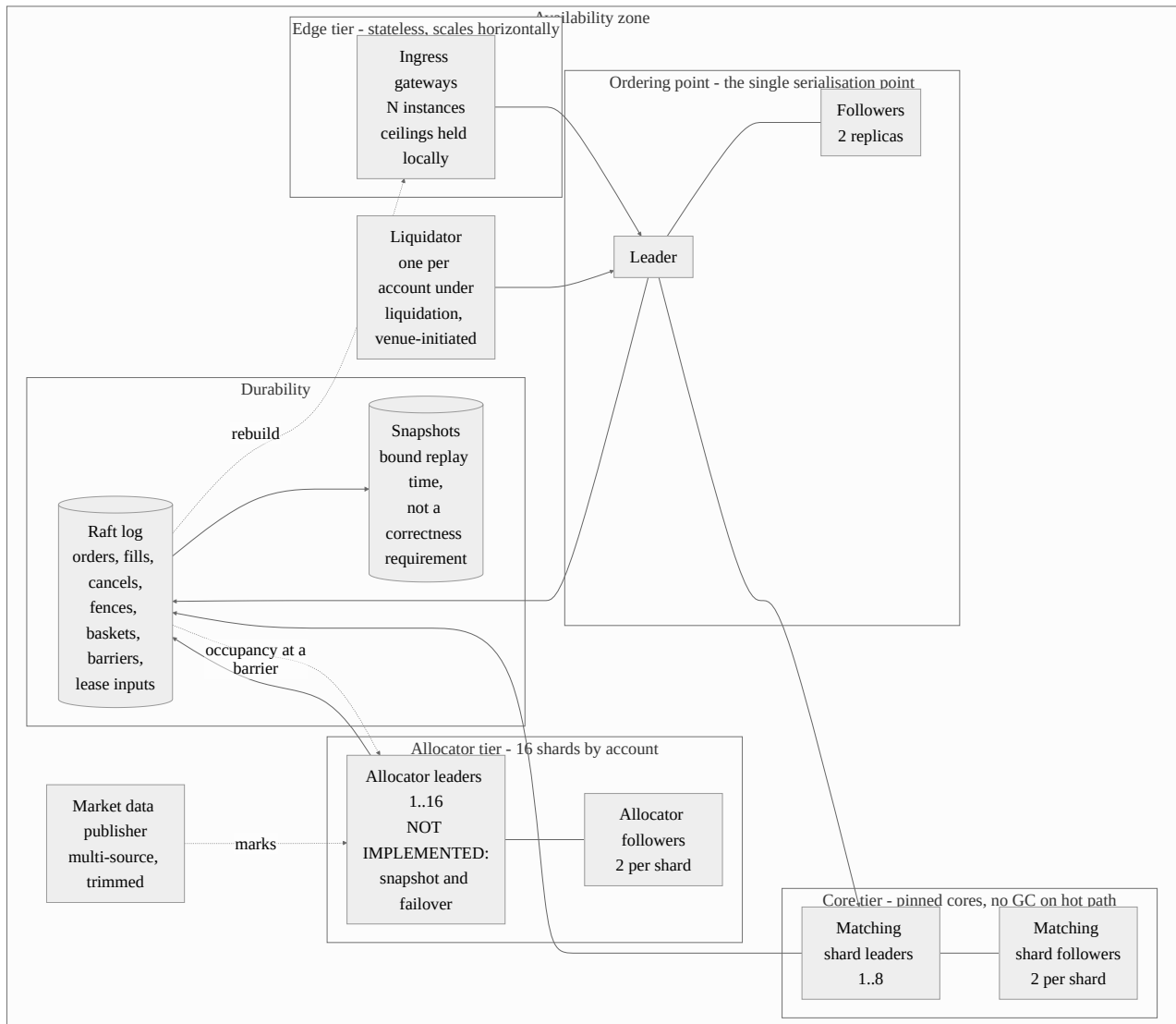


Figure 5: Deployment view. The target deployment; no part of it has been implemented or measured.

Flow	Model	Defence
Matching, per symbol	Single-writer total order	Price-time priority <i>is</i> a total order; anything weaker changes which client got the fill
Envelope consumption at a gateway	Local atomic within the gateway	Three integers and a vector owned by one gateway; nobody else reads them during the term
Admission sequence per lease	Gap-free total order at the ordering point	It accepts only the next number for a lease. The seal, the barrier and the settlement figure all rest on the log holding every admission that lease produced
Authority binding	Registry at the ordering point, holder from the authenticated session	A lease id alone is a bearer token: without the binding, knowing one is enough to submit for any account (Appendix C.3)
Fill terms	Checked at the ordering point against terms recorded at admission	A fill reported by a component that could be compromised is not evidence. Band, fee cap, direction, over-fill and identity are decided there; a refused fill writes nothing
Lease issuance and generation	Single allocator per account, linearisable	Two issuers could double-issue against the same equity. The one place the design cannot weaken
End of authority	Fence at the ordering point, linearisable	A clock comparison is not evidence a partitioned holder has stopped; a fence is. It need not be delivered to the holder
Marks into the allocator	Bounded-stale	Staleness costs capacity in both directions and, inside the grid, not safety — for two reasons that are both needed: the add-on is reserved against G^+ , the highest gross the grid reaches, and R already covers the equity the account loses at any scenario in it
Position feed into the allocator	Bounded-stale, eventually consistent	A fill the allocator has not seen was admitted under a lease and is inside that lease's absolute ceilings. Until a terminal ordered reconciliation it stays charged to that holder
Holder occupancy	Over-approximated per holder while any is live; compacted from the log once none is	Per-holder figures are summed and do not net, which is necessary while an unreachable holder may still be acting (§5.4)
Audit journal	Durable append-only, linearisable per shard	Replay must reproduce the decision exactly, which fails if entries reorder
Dashboards, ledger balances, retry handling	Eventually consistent; fold of the journal; at-least-once with end-to-end idempotency	A stale capacity figure drives no decision the system will not re-check; exactly-once is not a transport property (§5.3)

The row worth arguing is the position feed. A stale feed cannot under-state the requirement, and the reason is **not** that lag produces a smaller ceiling — neither the mechanism nor reliably true. It is that the ceilings are absolute rather than incremental and the accounting is monotone: the allocator never releases a holder's occupancy on the strength of not having heard from it.

3.2 CAP position of the matching core

Unchanged from the running case: during a partition that costs the leader its majority, the matching core stops accepting orders rather than risk divergence. Halting is embarrassing; double-executing trades is existential.

3.3 CAP position of the admission plane

The relevant partition is **between a gateway and the allocator, with the ordering point still reachable**. A gateway cut off from the ordering point cannot reach a book at all and is not serving in any sense.

During a gateway-allocator partition, the gateway keeps admitting inside the ceilings it already holds until its term ends, and then admits nothing. It does not fall back on a local judgement that an order reduces risk.

An earlier version of this document said gateways keep accepting risk-reducing orders. That is withdrawn: c9 is an order

that lowers one gateway's requirement and raises the account's, by closing a leg whose hedge is held elsewhere. A gateway does not have the account's portfolio and cannot tell the two apart.

Three consequences.

Availability is bought before the partition, not during it. The gateway serves because it was handed a ceiling solved to be safe on its own; when the term ends, availability ends. The term length *is* the availability budget, and it is the same number as the tightening latency of §1.5.

When the term ends, all client order flow through that gateway stops, closing orders included. The liquidator is venue-initiated machinery, not a client-facing close-only API. What the design preserves under partition is **the venue's ability to bound its own loss**, not the client's ability to exit.

A client-facing reduce-only path is possible and not built. It needs the same account-level check the liquidator performs, so a central component the gateway can reach — and if that is reachable, the account is not partitioned in the way that matters.

3.4 Where the design refuses to weaken

1. **One issuer per account.** Two allocators can issue two sets of ceilings against the same equity; each is individually valid, so no downstream check recovers.
2. **Authority ends at the ordering point, not on a clock.** If a fenced lease can still admit, the seal names a count that is already wrong and every figure computed from the log is computed over an incomplete one. E7's `no_fence` arm draws 7,790 on the insurance fund against 0 in the base run, takes 75 transfers instead of 24, and cannot settle at all.
3. **Append-only, ordered audit.** Reordering makes an admitted decision unreproducible, which is the whole of what §6.4 offers a supervisor.

Everything else is chosen at the weakest model that survives.

4. Data and storage design

4.1 Engine per store, chosen from the access pattern

Store	Access pattern	Engine	Why not the obvious alternative
Order books	Top ticks at 100k/s; cancel by ID O(1)	In-memory price-level array, FIFO per level, index by order ID	A tree of queues costs a cache miss per hop; at 100 µs the misses are the budget
Scenario vectors and gross totals	Update per order, per (account, gateway)	Flat array, ≈ 64 MB resident	A revaluation per order is pointer-chasing at the top of book
Account positions and equity	Read per issuance; written on every fill	In-memory in the allocator shard, snapshotted	A row store serialises the hot accounts, which are the market makers
Log, journal and ledger entries	Sequential append at ≈ 21 MB/s (§2.7)	Append-only segments on NVMe	Nothing is updated in place
Balances	Fold of the journal, materialised	In-memory hash, snapshotted with the journal offset	Balances as source of truth is the running case's anti-pattern, Part 3 §7
Historical balance-at-time-T	Rare, analytical	Columnar store fed by change data capture	Otherwise an analytical workload sits behind the trading lock
Scenario grid, add-on parameters, gateway weights, band and fee policy, credit version, authority bindings	Read on every derivation and replay	Versioned data on the log, activated at a sequence	As configuration they make replay non-deterministic

The last row is specific to this design: those values look like configuration an operator would tune, and are not, because a gateway derives its ceilings and its refusals from them and a replay must reproduce both.

4.2 Accounting model

Double-entry, with conservation enforced at write time: every journal entry's postings sum to zero per asset, so value is moved by code and never created. Accounts are keyed (`ownerId`, `assetId`, `accountType`) over `USER_AVAILABLE`, `USER_MARGIN_HOLD`, `USER_UNREALISED`, `EXCHANGE_FEE`, `INSURANCE_FUND`, `EXCHANGE_HOT`, `EXCHANGE_COLD`, `SUSPENSE` and `EXTERNAL_SETTLEMENT`. `USER_MARGIN_HOLD` is the running case's `USER_HOLD` under cross-margin: there the hold is per order and released when that order resolves, here it is the account's single encumbrance and moves with the portfolio requirement. `USER_UNREALISED`, `INSURANCE_FUND` and `EXTERNAL_SETTLEMENT` have no counterpart in a spot venue.

Signs follow each account's normal balance — user and asset accounts debit, venue liabilities credit — and an entry's postings sum to zero once signs are applied. `EXTERNAL_SETTLEMENT` is a clearing account: value leaving the venue reduces both a liability and an asset, which is two debits and does not net, so it is booked in two entries that each sum to zero. An earlier version wrote it as one entry and violated its own rule.

Amounts are signed 64-bit integers in minimal units with 128-bit intermediates. No floats: they break determinism, which is what makes replicas agree and replays reproducible.

4.3 Leases, holds, and what actually follows

A lease is an authorisation. A hold is a posting. A lease never appears in the ledger: it is capacity to create holds, issued by the allocator, consumed by a gateway, ending with its term. A hold moves collateral from `USER_AVAILABLE` to `USER_MARGIN_HOLD` when a position opens. Conflating the two is how an authorisation becomes spendable.

An earlier version chained them into `sum holds <= sum consumed leases <= sum issued leases <= collateral`. That is withdrawn and does not hold: orders are checked against absolute worst-fill envelopes rather than each consuming an additive quantity, so “consumed leases” is not a sum, and a scenario requirement, a gross notional and an execution cost are not commensurable and do not compose into a ledger amount.

What does hold is three facts that meet at the account: every posting sums to zero per asset, enforced at write time; an account's equity is a cash-flow fold of the authoritative log, rebuildable independently of any live component (a14); and the admission theorem of §2.4 keeps the requirement inside *that* equity after any move the grid covers.

They do not compose into a proof that venue assets exceed liabilities: that needs the ledger implemented, the insurance fund sized, and moves outside the grid accounted for.

4.4 The order lifecycle as journal entries

Event	Postings
Order admitted	None. Admission consumes envelope, which is not money
Position opened by a fill	<code>USER_AVAILABLE -x / USER_MARGIN_HOLD +x</code> ; fee to <code>EXCHANGE_FEE</code>
Mark-to-market	<code>USER_UNREALISED</code> against the counterparty; sums to zero across both plus fee
Position reduced	<code>USER_MARGIN_HOLD -x / USER_AVAILABLE +x</code>
Liquidation basket	As a reduction; the counterparty is the venue (§5.4), and a shortfall draws <code>INSURANCE_FUND</code>
Withdrawal requested	<code>USER_AVAILABLE -x / SUSPENSE +x</code> , after the sequence in §6.2
Withdrawal confirmed	<code>SUSPENSE -x / EXTERNAL_SETTLEMENT +x</code> , and <code>EXTERNAL_SETTLEMENT -x / EXCHANGE_HOT -x</code> when the chain confirms

The first row is the one that matters: admission moves nothing, which is what lets it run at gateway speed, and why the envelope bound has to be sound — it is the only thing between an admitted order and an over-committed balance.

A withdrawal reduces equity, so capacity outstanding against the old figure has to stop before funds leave: **fence, reconcile, re-issue against the reduced equity, release**. Bumping the generation is not sufficient — a partitioned gateway keeps admitting inside its term regardless. §6.2 gives the slower alternative.

4.5 Write path, idempotency and reconciliation

Entry IDs are a hash of the trade sequence and leg index, so a replayed trade deduplicates. Assert-then-apply is atomic over the accounts touched; the ledger is a single-writer partition per asset class; negative balance is a write-time assertion. Reconciliation runs continuously: balances recomputed from the journal against the materialised view, sequence-range checks for gaps and duplicates, hot and cold mirrors against chain balances, and every account rebuilt from the log against the live ledger — which catches divergence between §4.3's two folds.

The ledger module is specified here and not implemented in the simulator, which models the envelope and account sides only: §4.3's three facts are established for the account, and NFR row 11's venue-level statement is an argument, not a checked property.

5. Failure and recovery

5.1 What has to survive a node loss

State	Owner	Size at the scale of §1.1	Rebuilt from
Order books	Matching shard	≈ 160 MB per shard at 8 shards	Snapshot plus log tail
Per-account positions, equity, ceilings, generation	Allocator shard	≈ 27 MB total	Snapshot plus log tail. Not implemented (§5.7)
Per-(account, gateway) scenario vectors and gross totals	Gateway	≈ 64 MB total	Derived; not snapshotted
Balances	Ledger	≈ 100 MB	Journal fold

The scenario vectors are a cache of a pure function of the order state, so they are rebuilt rather than restored: reconstructing 128 bytes from a handful of orders is faster than reading it from disk, and snapshotting them would double the snapshot and buy nothing.

5.2 What must be on the replicated log

§2.7 lists what an admission decision’s replayability puts on the log and why the ceilings are derived rather than logged. The requirement is stronger than the running case’s because three components must reconstruct the same facts without reaching each other, and two consequences carry into the rest of this section: **a snapshot bounds replay time and is never a correctness requirement**, and **replay is idempotent by log position** rather than by order ID — which covers admissions but not fills or cancels, and r2 caught that.

5.3 The idempotency chain

The full nine-step chain is Appendix C.1. Its shape:

- **the client** retries on the same client order ID; a timeout is an unknown outcome, not a failure;
- **the gateway** submits under `(lease_id, admission_seq)`, and the ordering point takes only the next number for that lease, so the recorded sequence is gap-free — which is what later lets a seal cover every admission;
- **the ordering point decides first**. Only a fill it accepted is folded into the gateway and the account; an earlier implementation called the gateway first and moved state the authority then refused;
- **a fill** is refused, writing nothing, on a reused identifier with different figures, an unknown or cancelled order, the wrong direction, an over-fill, a price outside the recorded band, or a fee above the cap; **a basket** is one record under one identifier, idempotent on retry; **the matching shard** applies a client order ID at most once.

Two costs stated rather than hidden. A retry routed through another gateway is a new admission attempt there and may consume envelope, released at the next issuance. And a cancel *request* releases nothing; only an acknowledgement recorded at the ordering point does, which produces the two failures below.

5.4 Fencing, liquidation and settlement

An account cannot reach a requirement above its equity through any move the grid covers; that is what the closure reserves for. **A liquidation is therefore an event outside the model, the credit event has already happened by the time anyone sees it, and the mechanism is limiting a loss rather than preventing a violation**. What it limits is the draw on the insurance fund once the account’s own equity is gone.

Three things stop, in three places

What	Where	Cost
New admissions	a fence at the ordering point	one write; no gateway need be reachable
Resting orders filling	a cancel acknowledged at the ordering point	a round trip per order, and it can fail
The position moving	trading it out	as long as the unwind takes

The unwind is an internal transfer Reducing one leg at a time does not work: on a hedged book, closing one leg while its offset stays put raises the account’s requirement — c9 with the liquidator in the gateway’s place. 13 measures the requirement going $0 \rightarrow 2,000$ on a single-leg proposal and staying at 0 on a proportional basket.

That forces a basket, and matching is sharded by symbol, so **a basket spanning several symbols cannot fill atomically across several books**; a partial fill on one shard with none on another leaves the account somewhere the check never approved. This design does not assume an atomicity the matching path lacks and does not route baskets through it. The liquidator prices the whole basket against the venue’s own marks, inside the same band and fee cap an ordinary fill is held to, and the ordering point commits it as one record: either the record is there and the whole basket happened, or none of it did.

The cost is that the venue is the counterparty. Risk moves to the venue’s own book and, past the account’s equity, to the insurance fund. Venue-side portfolio risk and capital limits for that book are not modelled in this document.

Two ways a cancel fails

Failure	What the log holds	What the settlement must do
Acknowledgement recorded, notification to the gateway lost	the cancel	release the order; only the local view is stale
Matching side never confirmed	nothing	keep the worst-fill reservation and the execution-cost reserve

Fencing does not help in the second case: it stops new admissions and does nothing to a resting order. Both arms of E7 show one order live at the end; the settlement figure is $(0, 0, 0)$ for the recorded cancel and $(120, 2232, 9)$ for the unacknowledged one.

Two kinds of release **A seal releases one lease** — portable evidence a holder carries to the allocator, refused unless the lease is fenced and the seal’s terminal sequence matches, with the figures taken from the log rather than from whoever asks.

A barrier compacts the whole account. Per-holder occupancy is summed and does not net — necessary while any holder may still be acting, and badly wrong once none is: after a liquidation each ingress lease reconciles to its own gross leg, the offsetting legs sit under the liquidator, and an account holding nothing looks fully occupied. The account-wide path uses no seal, because the allocator reads the same log the seal was cut from, so an undelivered seal freezes nothing.

A seal releases one lease. A globally fenced, ordered account barrier permits account-wide compaction.

Compaction requires six conditions, none supplied by the caller: issuance stopped for the account; every lease it ever held fenced, including every incarnation and the liquidator’s own basket authority; a barrier watermark at which the recorded sequence is gap-free and no admission was made under an unregistered lease; the occupancy rebuilt from the log at that watermark; a credit-version compare-and-set on the install; and issuance resumed only afterwards. Appendix C.2 states them in full. E7 refuses `no_fence` with two ingress leases live and `liquidator_authority_live` with the liquidator’s lease live, and settles `seal_undelivered`.

What the delay costs E6 decomposes the equity change exactly and asserts the identity on integers with no tolerance:

```
ending equity == trigger equity + drift - slippage - fees
```

Across the delay sweep execution cost runs $2,522 \rightarrow 6,486$ while drift runs $16,444 \rightarrow 229,708$. **The two things the mechanism controls — new admissions and the execution cost of unwinding — are bounded. Market drift is not.** In E6 it grows linearly with the delay, but that is a property of the constant-rate price path used, not a result. The unwind’s cost sits inside the bound taken over the reachable position **when authority ends**, and outside any bound taken when the shortfall was detected, because everything admitted during the detection delay is inside the first and outside the second. In the unfenced arm no such moment exists and the cost exceeds the bound by 1,562.

The required-buffer figures E6 reports are measurements of one configuration, one seed and one price path. They are not a bound.

5.5 Recovery-time arithmetic

State is not one snapshot: the tiers of Figure 5 fail independently and are sized separately.

Tier	Snapshot	Log tail over 5 min	Conclusion
Matching shard	≈ 160 MB (one shard of eight)	orders and fills for that shard’s symbols	Dominated by the snapshot read, then replay
Allocator shard	≈ 27 MB total across sixteen shards	lease inputs, ≈ 8 MB/s	Small enough that replay dominates
Gateway	none — its state is derived (§5.1)	admissions, fills, cancels for its own lease	Rebuild is pure replay

The one figure that can be checked from this document is the replay volume:

5 min at ≈ 21 MB/s (\$2.7) = 6.3 GB;
 at an assumed 10 $\times$ live replay rate, the log tail takes about 30 s,
 excluding snapshot fetch and leader election.

Two caveats the arithmetic does not carry: 21 MB/s is a lower bound, counting order commands and lease inputs but not fills, cancels, fences, baskets, framing or any replication factor; and the 10 $\times$ replay rate is **assumed**, not measured (§8.1).

Warm failover is a design target, not a result. Row 8's three seconds would be election plus commit catch-up, and neither replication nor leader election is implemented (§5.7). What *is* established is the property failover would need: E4's 3,642 injected crashes show snapshot plus replay reproducing the state the whole log implies, zero equivalence failures, including 2,364 mid-partial-fill and 819 stale-snapshot cases. During such a window gateways admit inside the ceilings they hold and, once terms expire, nothing at all — so a failover longer than the shortest outstanding term leaves accounts with no ingress.

5.6 Zero-downtime upgrade

Anything a decision is derived from is versioned data on the log, activated at a sequence (§4.1), so a replay uses the values in force then. Upgrades are rolling with the state machine version pinned per record, and an operator cannot tune a derivation mid-session — a real operational loss, and the price of NFR row 10.

5.7 What is not covered here

This is a deterministic simulator, not a deployment. Everything measured runs in one process against an in-memory ordering point. A replicated ordering point, allocator high availability, leader election and real distributed deployment are *designed* and *not built* (Figure 5); the recovery results establish the property replication would need, not that replication works. The allocator alone cannot rebuild itself from the log, and that is not implemented either.

The liquidation transfer moves risk onto the venue (above): venue-side capital limits and insurance-fund sizing are outside this document, as are the waterfall beyond the unwind, the replay rate of §5.5, and cross-datacentre replication.

6. Security and threat model

6.1 Trust boundaries and the trusted computing base

Boundary	If the far side is fully compromised
Client $\leftrightarrow$ gateway	The account's own ceilings. Ceilings are per account, so one client cannot consume another's
Gateway $\leftrightarrow$ ordering point	Inside the trusted computing base. See below
Ordering point $\leftrightarrow$ everything	Total. Every safety claim rests on it
Liquidator $\leftrightarrow$ ordering point	The account it is liquidating, in full
Market data $\leftrightarrow$ allocator	How much capacity is solved for. Not the admission check itself
Allocator $\leftrightarrow$ gateway	An account's whole capacity
Chain $\leftrightarrow$ custody	Venue assets. Unchanged from the running case
Operator $\leftrightarrow$ ledger	Nothing; no write path. Detection is reconciliation, not prevention

The gateway is inside the trusted computing base. An earlier version of this document claimed its blast radius is bounded by the leases it holds. That is withdrawn. Nothing downstream re-derives the envelopes: the ordering point checks that an admission is the next number for a live, correctly bound lease and that a fill matches the terms recorded at admission; it does not recompute the worst-fill figures the gateway compared against its ceilings. **A gateway that lies about its own envelope arithmetic is believed.**

What a compromised gateway cannot do — each pinned by a test:

- act under a fenced lease, or under a lease id the allocator never minted (t3, t4);
- use another account's lease, or another holder's, or submit with no authenticated session (t1, t2);
- commit a liquidation basket under an ingress lease (t5);
- fill outside the band or above the fee cap recorded at admission, fill in the wrong direction, over-fill, or land the same fill twice (d4–d6);
- hide an admission, because the ordering point takes only the next sequence number for its lease.

What it can do, stated without softening: **a compromised gateway is bounded by neither its ceilings nor its own term.** It can submit arbitrary quantity for every account whose lease it holds, and it will not stop at its expiry, because the ordering point does not enforce terms — it has no clock it can compare against an expiry set elsewhere (ADR-6). A term bounds an *honest* gateway that has lost contact. The only thing that stops a dishonest one is a fence committing at the ordering point.

So the mechanism bounds the **scope** of the damage — which accounts, which authority kind, which holder — and does not bound its **magnitude**. The exposure window is detection latency plus fence-commit latency, and nothing in this document

measures either. Closing this would mean re-deriving the envelopes at the ordering point, which puts the per-order margin computation back on the single-writer path — the thing §2.2 exists to avoid. That trade is not made here; it is the residual.

The liquidator is inside it too, on its own account. Its orders are checked against the merged account rather than a ceiling (c9), so no ceiling bounds it. The non-increase test bounds the *risk* it creates — neither merged envelope may rise — but permits unlimited churn, and churn costs execution. Its authority ends at the ordering point like any other's, and the barrier refuses to run while it is live (t5, l11): containment of duration, not of authority.

The market-data path carries no authority in the running case. Here marks set equity, the scenario displacements and $G+$, so a wrong value changes how much capacity is solved for — weaker than an earlier draft claimed, since the admission check reads no market state, but it is the exposure in §6.3 A1.

6.2 Who can move money

Three ways, none of them the margin path: a trade, journalled by clearing; a deposit or withdrawal, frozen then reviewed then gated then signed once per withdrawal ID; and a liquidation or insurance-fund draw, journalled like any other posting. No component writes a balance.

A withdrawal reduces equity, so outstanding capacity against the old figure must stop before funds leave. Two orderings work and differ in latency, not safety: fence, reconcile, re-issue, release — immediate, at the cost of a fence round trip; or wait for the terms to end, then re-issue and release — free, binds within the term. Releasing first is not available.

6.3 Business-logic abuse cases

A1 — Overstating equity Every ceiling is solved against E_0 , and nothing downstream re-derives it. An account reporting more equity than it has buys a ceiling it cannot carry.

E5 measures it. An account that forgets a realised loss reports 92,000 where it has 42,000, is issued 46,000 instead of 21,000, admits 230 orders instead of 105, and ends 4,000 above equity. At the binding point the breach tracks the overstatement approximately one for one, subject to lot rounding: overstating by 10,000 produces a breach of 10,000, while 1,000 produces 800, because the ceiling cannot move until the overstatement buys a whole lot of requirement. **The factor of two is a closure, not a margin against a misreported account;** an earlier draft that read it as a 64% tolerance was reading unused workload slack.

The exposure is to anything that moves E_0 : a suppressed mark, a lost fee, a realised loss not yet folded in. The defences are §4.3's cash-flow identity, the account being an independently rebuildable fold of the log, and multi-sourced marks. None is a proof; a compromised equity path is a compromised mechanism.

A2 — Routing to concentrate an account's flow A gateway prices nothing globally: it compares absolute figures against its own ceilings and gets no credit for offsets held elsewhere, so the account is charged the sub-additivity gap of §2.3. Concentrating onto fewer gateways the positions that *offset each other*, or whose losses peak in *different* scenarios, shrinks that gap and buys real capacity — not by concentrating market risk, but by making the decomposition less conservative.

This is a **capacity-fairness** problem, not a solvency one: the requirement is still bounded, but a client who can influence routing — by picking a gateway, or retrying until it lands where it wants — gets more usable capacity than one who cannot.

Mitigations, none implemented: account affinity and controlled failover keep offsetting legs together and so shrink the gap; usage-aware weights do not shrink it at all, they only reduce stranded capacity on gateways an account is not using.

A3 — Stalling a liquidation Leaving orders the matching side never acknowledges cancels for keeps the account in the fenced state indefinitely: the settlement reserves for them rather than releasing them (l13), so the capacity stays occupied. The venue's exposure is bounded — the reservation is exactly the worst-fill figure — but the account cannot be closed out and re-opened. Detection is §8.2's fenced-but-not-settled alert; there is no mitigation in the design. Fee-cap gaming through many small fills is the fourth case and is bounded by the per-lot cap the ordering point enforces (d4).

6.4 Regulatory posture

What this design offers a supervisor is **reproducibility of what was admitted**: every admitted order is on the ordering point's log with the lease, the holder and the terms it was held to, and the values the decision was derived from are versioned on that log (§4.1), so an admission from three months ago can be recomputed rather than described.

It does not yet offer the same for a **refusal**. A gateway that refuses an order on its own envelope arithmetic sends nothing to the ordering point, so there is no record to replay. NFR row 10 is written as a target for that reason, and the gateway refusal journal that would close it is designed and not built.

What it does not offer is a guarantee that clients can always exit. The venue can act on an account it can no longer margin (§5.4), but a client whose gateway's term has ended cannot close through it and there is no client-facing close-only path (§3.3). A supervisor asking whether clients can always exit should be told no; §7 records that the wider claim was considered and not built.

6.5 Detection

Six signals, all paged on and listed in §8.2: equity divergence between what the allocator solved against and what a rebuild from the log implies; ceilings summing above the solve; a gateway's figures above its ceilings; a lease fenced but not settled beyond a bound; mark divergence across sources; and any authority-binding refusal, which should be zero. The gap this list cannot close is §6.1's: nothing detects a gateway that lies about arithmetic nobody re-derives, until the account's own figures move.

7. Trade-offs and alternatives

Each decision with the alternatives considered and the reason each lost. Where a number decided it, the number is given. The reversals are summarised here and recorded in full in Appendix A.

ADR-1 — How the account-level invariant is enforced pre-trade

Decision. Divide the account's capacity into per-gateway ceilings checked locally against absolute figures.

Alternative	Why it lost
Lock the account for the check	The only <i>exactly</i> correct option — full offset, no conservatism — and it puts the running case's fee-account hot-row problem (Part 3 §4) on every order. Market-maker accounts are the hottest objects in the venue
Fixed per-gateway sub-limits, no offset	Safe, trivial, and it deletes the product. The value forgone is exactly the sub-additivity gap $\sum_g R(P_g) - R(P)$ taken to its maximum
Admit optimistically, repair after	Violates the rule that a balance never goes negative. A venue proving assets $\geq$ liabilities at any instant cannot have a window where the proof is pending

Cost. A gateway gets no credit for offsets held elsewhere, so the account is charged the sub-additivity gap. §6.3 A3 records the routing distortion that creates.

ADR-2 — Flat ceiling for the term, or a schedule over market states

Decision. A flat ceiling. **This reverses an earlier decision**, and the reason is short: a lease cannot remove a position it has already admitted, so capacity that shrinks with the market restricts the *next* admission and does nothing about the exposure already created. A flat ceiling at the solved level is equally safe and admits at least as many orders as any decaying one with the same start. The figures that argued otherwise came from a mechanism that charged the increment rather than the absolute envelope (c1), and are withdrawn with it. Appendix A.1 has the full reversal; Appendix A.2 keeps the abuse cases the schedule created.

What survives. A gateway evaluating a shrinking curve can notice locally, on a market-state tick with no order present, that its consumption is higher than the venue would like. That is a trigger, not a capacity mechanism, and its value is unmeasured.

ADR-3 — Where gross notional is measured

Decision. Two figures (§2.3): the requirement uses G_k , gross at the marks of the realised scenario; the reserve uses G_+ , gross at the highest mark the grid reaches.

Alternative — one figure at the current mark. Unsafe. A is a function of gross, gross depends on the mark, and a lease admits for a term over which the mark moves.

Reserve measured at	Lots admitted	Requirement after the move	Equity after	Over by
the issuance mark, 1000	296	1,320,871	938,728	382,143
G_+ , at mark 1200	249	942,615	948,457	inside by 5,842

Cost. 47 lots of 296, about 16% of capacity in that configuration — the price of making both terms maxima over the same scenario set.

Scope. G_+ is an upper bound on the per-scenario maximum in every case. It is *tight* only with one factor and non-negative loadings, where the gap is the rounding, at most one minor unit per lot (m4a). With signed loadings the symbols peak at different scenarios: m4b gives 24,000 against a best single scenario of 20,000. The bound stays safe; the tightness claim does not generalise.

ADR-4 — What goes on the replicated log

Decision. The lease inputs — scale and weights — one record per changed account per issuance. Each gateway derives its own three ceilings.

Alternative — log the per-gateway ceilings. Rejected on bandwidth: 10^4 accounts $\times$ 5 gateways $\times$ 64 B $\approx$ 32 MB/s at a 100 ms cadence, against an order stream of $\approx$ 12.8 MB/s. The inputs are $\approx$ 8 MB/s. Two and a half times the order traffic to carry a derived value did not survive the arithmetic.

Rule extracted. Log the inputs a derived value comes from, not the derived value, unless the derivation is not deterministic — the same argument the running case makes for balances being a fold of the journal.

Cost. Everything in the derivation becomes versioned state-machine data (§4.1), so it cannot be tuned mid-session.

ADR-5 — How the allocator is partitioned

Decision. By account, sixteen shards.

Alternative — by symbol, matching the core. Rejected: margin is an account-level quantity, so shards would combine partial views to produce one account’s ceilings, reintroducing the coordination the design exists to remove. The two partitionings are orthogonal (§2.1).

ADR-6 — What ends a holder’s authority

Decision. A fence at the ordering point, with the lease term as the fallback for holders nobody is asking about, and a registry binding each lease to an account, a holder and an authority kind.

Alternative	Why it lost
Compare the order’s generation with the lease’s	A stale gateway and a stale order agree with each other, so the gateway keeps spending a replaced allowance. Corrected to “a gateway that has seen a higher generation refuses to serve”, which is necessary and not sufficient
The allocator compares its clock against the expiry	A partitioned gateway’s clock may be behind, so concluding the term ended concludes nothing about whether the holder stopped (c11)
The holder reports that it has stopped	The interface defect that kept recurring: a correct seal paired with an optimistic usage claim was accepted until <code>release</code> computed the figure from the log itself, and the same mistake reappeared in the first account barrier
Treat the lease id as the authority	It was a bearer token: any account, any claimed holder, until an external review demonstrated it (t1)

Cost. The ordering point has no clock it can compare against an expiry set elsewhere, so it does not enforce terms. An honest gateway is bounded by its own term; a Byzantine one only by the fence. §6.1 states that rather than claiming otherwise.

What we deliberately did not build

- **A liquidation waterfall.** The trigger, the fencing and the unwind are here; who absorbs a shortfall is a separate document.
- **The mark-price pipeline.** Marks set equity and every ceiling is solved against equity, so we would not claim the capacity control is sound without it (§6.3 A1).
- **A client-facing reduce-only path.** §3.3 says why, and what it costs.
- **Replication of the ordering point, and allocator failover.** Figure 5 draws the target deployment and labels it unbuilt.
- **Anything that identifies the agent behind an order.** The mechanism is capacity control, not surveillance.

8. Operations*8.1 The first chaos experiments*

Measure the replay rate, because §5.5 assumes it at ten times live and nothing else in that arithmetic is assumed. Cold-start a node from a snapshot plus five minutes of log, time the rebuild, and compare the state hash against the live node — the comparison E4 makes 3,642 times. At or above 10 $\times$ the five-minute snapshot cadence stands; at 2 $\times$ it tightens to about a minute and nothing else moves, because a snapshot bounds replay time rather than correctness.

Second: partition a gateway from the allocator while it stays connected to the ordering point, and hold it past its term. This is the failure §3.3 is about. What should be observable is the gateway admitting inside its ceilings until the term ends and then admitting nothing at all, closing orders included. Partitioning a gateway from the *ordering point* tests nothing about margin: no order reaches a book at all.

Third: fence an account's leases without telling any gateway and confirm the ordering point refuses what follows. E7 counts 50 refusals in simulation; in a deployment this checks that the fence is a real serialisation point.

8.2 Alerts, business invariants first

1. **Sum of issued ceilings above what the condition solved for.** An arithmetic impossibility if the allocator is correct, so firing means it is wrong.
2. **A gateway's worst-fill figures above the ceilings it holds.** Same class.
3. **Requirement above equity outside a liquidation window.** The invariant the design exists to hold.
4. **Equity divergence** between what the allocator solved against and what a rebuild from the log implies, and **mark divergence** across sources. §6.3 A1 is the reason both page rather than sit on a dashboard.
5. **A lease fenced but not settled beyond a bound**, and any **authority-binding refusal**. The first is a stuck liquidation holding capacity; the second should be zero in normal operation and otherwise means a component is submitting under a lease that is not its own.

Alerts 1 to 3 are the invariants the simulator checks after every admitted order, promoted to production: the same predicate gates a test run and the live system, so a production failure is expressible as a failing test case. Latency percentiles, queue depths and replica lag are diagnostics and do not page.

8.3 Volatility-day playbook

Before. Pre-scale gateways; the matching core is partitioned, not elastic. Confirm every mark source is live and independent. Decide the term for the session: it is the tightening latency the venue accepts under partition (§1.5), trading availability against how fast a credit decision binds.

During. Nothing to tune. A tightening is a re-issue against lower equity, binding at the next issuance for reachable gateways and within the term for unreachable ones. To bind faster, fence the affected leases.

Liquidation is venue-initiated. The operator decides when to trigger, how aggressively to unwind, and whether to accept a stall: the unwind halves its basket fraction on a failed check and stops at one lot rather than forcing a reduction that would raise the requirement. A stall is a signal to widen the basket, not to push.

Halt and reopen. Cancels accepted, no new orders. On reopen, ceilings are re-issued against post-auction marks before order entry resumes, because the auction can move equity by more than the grid covers.

After. Reconcile per §4.5, and confirm every account that entered liquidation reached a barrier: fenced but never settled is the state that silently costs capacity.

8.4 What operations cannot do

- **Grant capacity by hand.** It comes from the condition of §2.4 or not at all.
- **Recall a lease from an unreachable gateway.** Fencing stops what it can still do; it does not remove what it already did.
- **Release an account's occupancy without a barrier** — not by clock, not by a holder's report, not by an operator's judgement that the holder is gone.
- **Write to the ledger.** Corrections are journalled, dual-approved commands.

The middle two look like an operator denied a manual override mid-incident. Both were tried and produced c8, c11 and c12: an operator who can declare a holder finished can hand its capacity to a replacement while it is still trading.

9. AI-disclosure appendix

9.1 How AI was used

An AI assistant was used throughout as a co-author: proposing mechanism, writing the simulator, deriving the Fermi estimates, drafting these sections. Every experiment was executed rather than described, and because the simulator is integer-only and seeded, runs on Python 3.12.3/Linux and 3.13.5/macOS agree except for E3's wall-clock figures (`results/PROVENANCE.md`).

Below is what the rubric asks for: what was rejected, and why.

9.2 Prompts that mattered

“Verify before building on it.” Applied to citations and to the assistant's own account of the design's risks; it surfaced that the risk named first was not the real one (9.3, item 1).

“Comments say what the code does, not what it is supposed to prove.” Applied to the simulator and every experiment script.

“Does the constraint actually bind?” Asked of the first E1 run, which reported zero violations because the budget was far above what the flow consumed. A zero from a slack constraint says nothing. E1 now carries a **binding trial** where the risk and debit envelopes reach 99% and the requirement is 49% of equity; only that zero is meaningful.

“Attack your own interface.” External hostile review was run each round against the code rather than the prose. It produced the authority-binding failure (9.3, item 6) and, late on, the discovery that a superseded experiment left on disk was passing on an empty sample.

9.3 Suggestions and claims rejected, and why

1. **That tiered maintenance margin breaks the decomposition.** Proposed as the leading risk. It is not: tiers are per contract and additive. What actually breaks decomposition is the portfolio-level add-on, which is super-additive.
2. **Abstracting the requirement into a general coherent-risk-measure formulation.** Rejected: §2.3 rests on exchanging a maximum with a sum, and burying that step inside a general functional makes the paper harder to defend, not stronger.
3. **Charging an order its marginal requirement.** A leg flipped from short to long leaves the increment unchanged while the account’s requirement moves to its maximum (c1). Replaced by absolute worst-fill envelopes.
4. **A price-conditional schedule as a safety and capacity mechanism.** Withdrawn with its evidence (ADR-2). A lease cannot remove a position it already admitted.
5. **Letting a gateway accept locally risk-reducing orders under partition.** Withdrawn: c9. Risk reduction is an account-level operation.
6. **Treating a lease id as authority.** Review demonstrated a valid lease id submitted under a different account returning success. Replaced by the registry and session binding of Appendix C.3.
7. **Reading the factor of two as a 64% tolerance for a misreported account.** Measured in a non-binding configuration. At the binding point the breach tracks the overstatement one for one (E5 Part B).
8. **Expiry releasing exposure, and a holder’s own report releasing it.** Both replaced by seals and the account barrier (c8, c11, c12).

9.4 What the numbers are, and are not

- **Current evidence:** E1–E7 and eleven test files — 16 lifecycle counterexamples, 6 worst-fill cases plus an exhaustive closed-form comparison over 4,000 books, 6 recovery, 14 liquidation, 6 repricing, 6 authority, 7 execution-cost and 14 ledger cases, and a fuzz over 10,060 admissions.
- E3’s absolute latencies support a **scaling** claim, not NFR row 2’s target.
- E6’s required-buffer figures are one configuration, one seed, one price path.
- The ledger of §4 is designed and not implemented; §4.3 establishes three facts about the account, not venue-level solvency.
- The replay rate of §5.5 is assumed, not measured (§8.1).
- §6.3 A3 is argued, not measured.
- Replication and allocator failover are designed and not built (Figure 5).
- Superseded results — E4’s schedule comparison, E5’s suppression sweep — are in *experiments/superseded/* and are cited nowhere as current.

9.5 Section ownership

TODO — to be filled in before submission; these must not reach the PDF. Every member presents their own sections and the panel may ask any member any question, so the split has to be one the group has rehearsed, not one assigned on paper.

§	Section	Owner
1	Business context and requirements	TODO
2	Architecture	TODO
3	Consistency map	TODO
4	Data and storage design	TODO
5	Failure and recovery	TODO
6	Security and threat model	TODO
7	Trade-offs and alternatives	TODO
8	Operations	TODO
9	This appendix, and appendices A–C	TODO

Module	Owner
Risk decomposition, allocator, envelope solve	TODO
Ordering point, authority binding, fencing and seals	TODO
Gateway, worst-fill envelopes, recovery	TODO
Ledger, liquidation, settlement barrier, evaluation harness	TODO

Appendix A — Decision history

This appendix holds what the main body no longer claims. It is here because a design document that quietly deletes its reversals is less trustworthy than one that records them, and because §9 has to disclose what was rejected as well as what was adopted.

Nothing in this appendix is current evidence. Where a figure appears, it was produced by an interface the mechanism has moved off, and the experiment that produced it now lives in `experiments/superseded/`.

A.1 What was withdrawn, and what replaced it

Withdrawn claim	Why it failed	What replaced it
A capacity schedule contracting with a published market state makes the mechanism safe	A lease cannot remove a position it has already admitted. Capacity that shrinks restricts the <i>next</i> admission, not the exposure already created	A flat ceiling for the term. The schedule survives only as a local operational trigger, with no capacity claim (ADR-2)
A state-contingent schedule recovers 3.8× the throughput of worst-case sizing	Measured on the incremental-charge mechanism, which did not bound the account's requirement at all	Utilisation is reported directly: the closure caps it near half of equity, and E1's binding trial reaches 99% of two envelopes at 49% of equity
Charging an order its marginal requirement is sufficient	A leg flipped from short to long leaves the local increment unchanged while the account's requirement moves to its maximum (c1)	Admission compares absolute worst-fill envelopes
Gross notional is additive across the partition	Two gateways can hold opposite legs in the same symbol; they net inside the account	Gross is sub-additive by the triangle inequality, and the add-on is evaluated once on the summed gross (§2.3)
Expiry releases the exposure a holder created	A term ends authority. The positions stay	Authority and committed exposure are tracked separately; only a terminal reconciliation or an account barrier lowers the second (§5.4)
A gateway may keep accepting risk-reducing orders during a partition	An order that lowers one gateway's requirement can raise the account's by removing a hedge held elsewhere (c9)	A gateway in quarantine admits nothing. Risk reduction is an account-level operation (§3.3)
A compromised gateway's blast radius is bounded by its leases	Nothing downstream re-derives the envelopes the gateway compared against	The gateway is inside the trusted computing base, and §6.1 says what it can and cannot do
A lease id identifies authority	It was a bearer token: any account, any claimed holder	Registered bindings and session-resolved holders at the ordering point (<code>tests/test_authority.py</code>)
The closure absorbs an equity overstatement of about 64%	Measured in a configuration where the ceiling was not binding	At the binding point the breach tracks the overstatement one for one (E5 Part B)
The lease term is the latency of every credit decision	Raising a limit takes effect at the next issuance	It is the enforcement latency of a <i>tightening</i> under partition (§1.5)

A.2 The abuse cases the schedule created

Retained verbatim because they are the record of what the market-data authority argument was, and because §6.1's weaker claim only makes sense against them. Under the current condition the admission check reads no market state, so A1 has no target on the admission path; A2 survives in the weakened form now recorded as §6.3 A1.

A1 — Replaying an older market state The shard evaluates its schedule at the market state it reads. An account able to make a shard read a stale, more favourable state obtains capacity the current state does not permit.

Mitigation: the shard evaluates its schedule at the most adverse state it has observed since the lease was installed, not at the state on the current message. A replayed lower state changes nothing.

Cost of the mitigation when the feed is honest: none. In E5 the ratcheted and unratcheted configurations are identical on an honest feed — 58 orders admitted, zero ticks with the requirement above equity. The defence only engages when the state goes backwards, which on an honest feed it does not.

A2 — Holding the published mark below the market An account able to suppress the published state, rather than replay an old one, is a different attack: the state never goes backwards, it simply fails to go forwards. The ratchet cannot see this, because there is nothing to ratchet against.

E5 measures it. The published state is held at 0 between ticks 20 and 50 while the checker uses the true state:

Configuration	Suppressed	Admitted	Ticks above equity, of 480
Scalar lease	no	36	236
Scalar lease	yes	36	236
Schedule, current state	no	58	0
Schedule, current state	yes	62	12
Schedule, ratcheted	no	58	0
Schedule, ratcheted	yes	49	5

Three readings. The scalar lease is indifferent to the attack because it never reads the state — it is also above equity 236 ticks out of 480 for unrelated reasons (§7). The schedule closes that, and in doing so opens A2: suppression buys 4 extra admissions and 12 ticks of exposure. The ratchet reduces the exposure to 5 ticks and costs 9 admissions under attack.

This is a partial mitigation and we do not present it as more than that. The residual belongs to the mark pipeline, not to the lease: multiple independent sources, a trimmed statistic across them, staleness detection on each source, and a bound on how far the published state may lag the sources. That pipeline is named in §2 and not designed in this document, which is a scope decision rather than an oversight.

A.3 The counterexample ledger

Sixteen lifecycle counterexamples (`tests/test_counterexamples.py`, c1–c16), six worst-fill cases (w1–w6), six recovery cases (r1–r6), fourteen liquidation cases (l1–l14), six repricing cases (m1–m5 with m4 split), six authority cases (t1–t6), seven execution-cost cases (d1–d7) and fourteen ledger cases (a1–a14). Each was written after a failure, most of them found by external review, and each is named in the section of the main body that depends on it. The pattern worth extracting is that **every one of them was found by a check that compared against something recomputed independently**, never by a check that trusted a figure a component reported about itself.

The three that changed the mechanism rather than fixing an implementation are c1 (absolute envelopes), c9 (local risk reduction is not global) and m1 (gross measured at one mark). The rest are lifecycle discipline.

Appendix C — Protocols in full

The main body states what each protocol guarantees and cites the test that pins it. This appendix carries the step-by-step form, for a reader checking the guarantee against the mechanism rather than reading the argument.

C.1 The idempotency chain, in full

End to end, for one client order. Every link is a named identifier and a decision point that has been tested rather than argued.

1. **The client** assigns a client order ID and retries with the same ID on timeout. A timeout is an unknown outcome, not a failure.
2. **The gateway** checks its three envelopes and, if they hold, submits to the ordering point under `(lease_id, admission_seq)` where `admission_seq` is the next number it has used under that lease.
3. **The ordering point** accepts that pair only if it is the next number for the lease, and only if the lease is not fenced. A retry carrying the same payload under the same pair succeeds again and records once; the same pair with a different payload is a conflict and records nothing. The gap-free sequence is what later lets a seal claim to cover every admission the lease produced.
4. **A fill** is submitted under a `fill_id` against an `order_id`. The ordering point refuses it — writing nothing and moving nothing — if the identifier has been used with different figures, the order is unknown or cancelled, the direction disagrees, the cumulative quantity would exceed what was admitted, the price falls outside the band recorded at admission, or the fee exceeds the cap for that quantity. A retry with the same figures lands once (`tests/test_execution_debit.py`, d4 to d6).
5. **The order of operations is fixed**: the ordering point decides first, and only a fill it accepted is folded into the gateway and the account. An earlier implementation called the gateway first, which moved state the authority then refused.
6. **A liquidation basket** is committed under a `basket_id` as a single record. A retry with the same payload is idempotent; the same identifier with different figures is refused and moves nothing (`tests/test_liquidation.py`, l7). A process that dies between the commit landing in the log and the transfer being folded in locally rebuilds from the log, and `applied_baskets` plus the ledger's fill keys make the reapplication land once (l14).
7. **Replay is idempotent by log position**. An earlier version relied on the order ID for that, which covers admissions and not fills or cancels, and replaying the same slice twice applied the fills twice (`tests/test_recovery.py`, r2).
8. **The matching shard** applies a given client order ID at most once, so no duplicate book action occurs regardless of how many times the order was admitted upstream.
9. **Clearing** derives ledger entry IDs deterministically from the trade sequence, so a replayed trade produces the same entry ID and is deduplicated (running case, Part 3 §4).

Step 2 is where this design differs from the running case, and it is a cost we state rather than hide: a retry routed through

another gateway is a new admission attempt at that gateway and may conservatively consume envelope there, though it cannot produce a duplicate book action. That consumption is released at the next issuance.

One asymmetry is deliberate and matters later. A cancel *request* releases nothing; only an acknowledgement recorded at the ordering point does, because until then the order can still fill. That produces two different failures which are not the same fact and are tested separately (§5.4).

Exactly-once is, as in the running case, end-to-end idempotency layered on at-least-once delivery. Nothing in the margin path changes that.

C.2 The settlement barrier, in full

A seal is portable evidence for releasing **one** lease: the ordering point issues it when a lease is fenced, naming the last admission it recorded, and a holder carries it to the allocator. The account-wide compaction does not use one, because the allocator reads the same log the seal was cut from.

A seal releases one lease. A globally fenced, ordered account barrier permits account-wide compaction.

`Allocator.settle` requires all of the following, and none of them is supplied by the caller:

1. the account is in `settling`, so no new lease can be issued for it. Doing this first is what stops a lease minted after the barrier from being authority the settlement did not account for;
2. every lease ever minted for the account is fenced at the ordering point — every ingress lease, every incarnation, and the liquidator’s own basket authority. `all_leases` is the set the barrier is taken over, not `authority`, because a lease that admitted nothing is still authority;
3. a barrier watermark `B` at which the recorded sequence under each of those leases is gap-free and no admission or basket for the account was recorded under a lease outside the set. `submit` and `commit_basket` already refuse anything but the next number, so this check should never fire; it is here because the settlement’s whole claim is that the log is complete, and an assertion that never fires is cheap next to a claim that is assumed;
4. the aggregate rebuilt from the log at `B`: filled positions, orders with no authoritative cancel acknowledgement, worst-fill risk, repriced gross reach, and the execution cost still ahead of the account;
5. installation under a credit-version compare-and-set, so a settlement computed at an older barrier cannot overwrite a newer one;
6. issuance resumes only after the install.

`E7` exercises the refusals as well as the successes: `no_fence` is refused with two ingress leases live, and `liquidator_authority_live` is refused with the liquidator’s lease live even though every ingress lease is fenced and the account is flat.

C.3 Authority binding at the ordering point

The ordering point holds `lease_id -> (account, holder, kind)`, registered by the allocator at issuance because the allocator is the single issuer and therefore the only component that knows the binding. The holder on a submission is resolved from the authenticated session and never read from the request body. A submission is refused as `unknown_lease`, `wrong_account`, `wrong_holder`, `wrong_authority_kind` or `unauthenticated` before any sequence check runs.

`kind` is `ingress` or `liquidation` and they are not interchangeable: an ingress holder that could commit a basket would be admitting with no ceiling at all, since the liquidator’s admissions are checked against the merged account instead.

Deliberately not checked here: the lease’s term. The ordering point has no clock it can compare against an expiry set by another component, so an honest gateway is bounded by its own term and a Byzantine one only by the fence (§6.1, ADR-6).

`tests/test_authority.py` pins six cases, including the one an external reviewer demonstrated against the previous interface: a valid lease id submitted under a different account, which used to return `(True, "ok")`.